

Preprocessing Explanation

1. Data Loading and Text Selection

The dataset was provided as a tab-separated text file containing an ID column and a sentence column. In the preprocessing stage, I loaded the file using pandas and used the `Sentence` column as the main text field. To make the program more flexible and able to work with new datasets in the same format, the script defaults to the second column if the `Sentence` header is absent.

2. Basic Text Cleaning (Text Normalisation)

All text was converted to lowercase, and unnecessary spaces and line breaks (e.g., `\n`, `\r`) were removed. Lowercasing ensures that identical words with different capitalisations (e.g., "The" and "the") are treated uniformly. Removing newline characters helps keep the text format consistent. This establishes a clean baseline, preventing the clustering algorithm from treating identical terms as distinct features due to formatting inconsistencies, thereby reducing feature noise.

3. TF-IDF Vectorisation

The cleaned text was transformed into a numerical matrix using TF-IDF (`TfidfVectorizer`), because K-Means cannot process raw text directly. I used English stop word removal (`stop_words='english'`), ignored very common words appearing in over 90% of documents (`max_df=0.90`), removed rare words appearing less than twice (`min_df=2`), and restricted the features to a maximum of 1000 (`max_features=1000`). These settings help reduce noise, lower sparsity, and improve computational efficiency.

4. Dimensionality Reduction via TruncatedSVD

`TruncatedSVD` was applied to reduce the 1000-dimensional TF-IDF matrix to 50 components. I tested both 100 components and 50 components. Although 100 components preserved nearly 40% of the original variance and gave a best silhouette score of 0.1538, 50 components performed better overall. Even though the explained variance dropped to 26.08%, the stronger reduction removed more noise and improved the best silhouette score.

Since TF-IDF matrices are usually sparse and high-dimensional, reducing the feature space made clustering more stable and efficient. As a result, the best silhouette score increased from about 0.03 (without SVD) to 0.2379.

Clustering Results Explanation

1. The Clustering Algorithm Used

The clustering algorithm used for this dataset is **K-Means**. It was implemented using the `scikit-learn` library. After preprocessing, each sentence was represented as a reduced numerical vector, so K-Means could be directly applied to group similar sentences. In my implementation, I used a fixed random state (`random_state=42`) and `n_init=10` to make the results reproducible.

2. The Optimal Value of K and Reasoning

Based on the quantitative results from the silhouette analysis, the optimal number of clusters is **K = 2**.

To determine the best number of clusters, I performed silhouette analysis for values of K from 1 to 10. Since the silhouette score is not defined for K = 1, I used 0.0 as a placeholder in the plot and calculated the actual silhouette scores for K = 2 to 10. The results showed that the highest silhouette score was obtained at K = 2, with a score of 0.2379.

Although the silhouette scores did not decrease at every step, the values after K = 2 were generally lower overall. For example, the score decreased from 0.2379 at K = 2 to 0.2278 at K = 3, then to 0.1931 at K = 4, and further to 0.1784 at K = 5. Some later values showed small fluctuations, and K = 10 increased again to 0.2044, but it still did not exceed the score at K = 2. This suggests that although a larger number of clusters may capture some more specific patterns in the data, it does not produce a better overall clustering result than K = 2.

I believe K = 2 is the optimal value because it gives the best balance between within-cluster similarity and between-cluster separation. In addition, choosing K = 2 keeps the model simpler and avoids dividing the dataset into more clusters than necessary. Overall, K = 2 was selected because it achieved the highest silhouette score and provided the clearest clustering structure among all tested values.